

Assignment - 8

Problem Statement :

→ In a networked environment, a shared printer receives multiple print requests from users throughout the day. To handle these requests efficiently and ensure optimal memory usage, the system implements a circular queue to manage the list of print jobs. Each print job is identified by a Job ID and document title. The print jobs are processed in the order they are received and memory is reused as jobs are completed.

Objective:

- 1) To implement a circular queue for managing print jobs efficiently using FIFO principles.
- 2) To perform queue operations such as enqueue, dequeue and display while handling overflow and underflow conditions.
- 3) To demonstrate efficient memory utilization by reusing vacant queue position through circular queue implementation in a networked printer management system.

Theory:

Simplification:

Platform:

64-bit open source linux or its derivatives

Open source C programming tool like gcc /
eclipse editor.

Pseudo code: ~

Initialization -

front $\leftarrow -1$

rear $\leftarrow -1$

Overflow

If $(\text{rear} + 1) \bmod \text{MAX} = \text{front}$
overflow

Underflow

If front = -1
underflow.

Enqueue -

If full $\rightarrow$ overflow

if front = -1 $\rightarrow$ front $\leftarrow 0$

rear $\leftarrow (\text{rear} + 1) \bmod \text{MAX}$

Or $(\text{rear} + 1) < \text{MAX}$

Dequeue -

If Empty $\rightarrow$ underflow
item $\in$ Q [front]

If front = rear
front $\leftarrow$ rear $- 1$

Else

front $\leftarrow$ (front + 1) MOD MAX

Display -

If Empty $\rightarrow$ "Empty"
i $\leftarrow$ front

Repeat

PRINT Q[i]

i $\leftarrow$ (i + 1) MOD MAX

UNTIL i = (rear + 1) MOD MAX

Time complexity:

Initialization: - $O(1)$

Overflow checking - $O(1)$

Underflow checking - $O(1)$

Enqueue - $O(1)$

Dequeue $\rightarrow O(1)$

Display - $O(n)$

Conclusion

- The circular queue efficiently manages print jobs by processing them in FIFO order while reusing memory through circular indexing.

FAQ'S

- 1) A circular queue is a queue in which the last position is connected to the first position. It follows FIFO principle.
- 2) It makes better use of memory by reusing the empty space created after deletion. It avoids the wastage of space that can occur in a linear queue.
- 3) A circular queue follows the FIFO principle, where the element inserted first is removed first.
- 4) Basic ~~basic~~ operations are:
 - Enqueue → Insert an element.
 - Dequeue → Remove the element.
 - Peek / Front → View the front element.

c) overflow occurs when the circular queue is full and no new element can be inserted

Condition: $(\text{front} + 1) \% \text{Max} = \text{front}$

d) Underflow occurs when an attempt is made to delete an element from an empty queue

Condition: $\text{front} = -1$

T) Task when queue needs the last position
It is steps ahead to the first position
because it is a circular. This is achieved using
the modulo (i.e. operation)

e) Application of circular queue:

- CPU scheduling
- priority scheduling
- buffer management
- network data buffering
- Traffic management system.

g) Both enqueue and dequeue take $O(1)$ time, because they involve only updating the front or rear position.

100

- 10) Circular queue can be implemented using array and two integer variables. The % operator is used to move front and rear circularly through the array:

ST
 10/10